

Laboratorio de Desarrollo de Soluciones Tecnológicas (LabDST)

Sesión 4 · De la simulación al paper

Dr. Jorge A. Lizárraga

Departamento de Procesos Tecnológicos e Industriales, ITESO, Tlaquepaque 45604, Jalisco, Mexico;
jorgea.lizarraga@iteso.mx

31 de agosto de 2026

Contenido:

- ▶ La receta de un resultado de simulación
- ▶ Figuras compuestas: figure + subfigure
- ▶ Ejemplo: una secuencia a dividir en 6 cuadros
- ▶ División en subplots con **trim**
- ▶ Código real: 6 subfiguras con **trim**
- ▶ Resultado visual equivalente (compilado)
- ▶ Regla 1: sin fondo en renders y capturas
- ▶ Regla 2: el marco del software, ¿sí o no?
- ▶ Reglas y consideraciones de figuras
- ▶ De código a pseudocódigo
- ▶ Agregar la librería al documento
- ▶ Anatomía de un algoritmo en $\text{\LaTeX}$
- ▶ Ejemplo 1 – Saturación del ángulo del servo
- ▶ Ejemplo 1 – pseudocódigo y su fuente en $\text{\LaTeX}$
- ▶ Ejemplo 2 – Bucle de simulación
- ▶ Ejemplo 2 – pseudocódigo y su fuente en $\text{\LaTeX}$
- ▶ Ejemplo 3 – Cinemática inversa de un módulo SOFIA
- ▶ Ejemplo 3 – pseudocódigo y su fuente en $\text{\LaTeX}$
- ▶ Reglas y consideraciones de pseudocódigo
- ▶ La receta completa, en una subsección
- ▶ Asignación

La receta de un resultado de simulación

Una simulación que corre en su computadora **no es todavía un resultado de investigación**: solo lo es cuando queda documentada en el artículo de forma que otra persona, sin ver su pantalla, pueda entender qué se simuló, cómo se calculó y qué se observó. Hoy veremos las tres piezas que convierten una simulación en una subsección del paper.

Las tres piezas de una subsección de resultados:

- 1. Una figura compuesta** (`figure` + `subfigure`): una secuencia o comparación de imágenes de la simulación, correctamente recortada y etiquetada.
- 2. Un pseudocódigo** (`algorithm` + `algorithmic`): el modelo o algoritmo aplicado, en notación matemática, no en el lenguaje en que lo programaron.
- 3. Un texto breve**: el párrafo que narra la figura y el algoritmo, citando ambos **antes** de que aparezcan.

! Ninguna de las tres piezas funciona sola: una figura sin texto que la explique, o un pseudocódigo que nadie referencia en el texto, quedan “suelos” y un revisor los marcará como incompletos.

Figuras compuestas: figure + subfigure

Cuando una simulación se documenta con **varias imágenes relacionadas** (una secuencia en el tiempo, distintas vistas, antes/después), **no se insertan como figuras sueltas**: se agrupan en una sola figura compuesta, para que el lector las interprete como una sola historia.

Dos niveles de referencia:

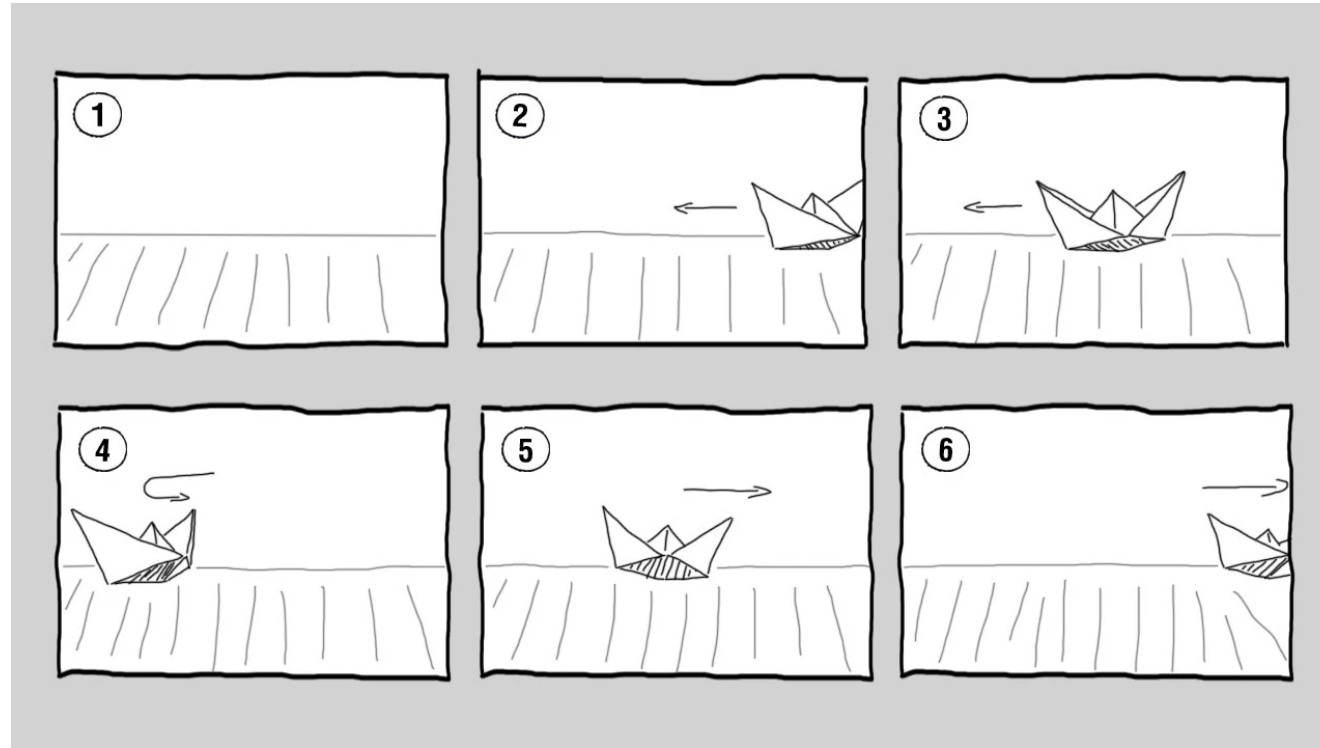
i

- ▶ **Referencia local:** `\label{}` dentro de cada `\subfigure`, para citar un cuadro específico: “*Fig. 2(c) shows...*”
- ▶ **Referencia grupal:** `\label{}` después del `\caption` de la figura completa, para citar el conjunto: “*Fig. 2 shows the full sequence...*”

- (1) Toda la secuencia vive en **un solo** `\begin{figure}...\end{figure}`.
- (2) Cada cuadro es un `\subfigure[leyenda]{...\}` con su propio `\label{}`.
- (3) Un `\caption{}` y un `\label{}` globales cierran la figura.
- (4) Ambos niveles se citan en el texto, **antes** de mostrar la figura.

Ejemplo: una secuencia a dividir en 6 cuadros

Usaremos esta imagen (un único archivo, `secuenciadefotos.png`) como si fuera una captura de su simulación con varios instantes de tiempo en un solo lienzo. La técnica que sigue sirve igual para 6 fotogramas de su video de seguimiento, 6 gráficas de MATLAB o 6 vistas de un módulo SOFIA.



! No hace falta recortar la imagen manualmente en un editor: L^AT_EX lo hace por ustedes con `trim` y `clip`, cuadro por cuadro, sin duplicar el archivo.

División en subplots con `trim`

La imagen mide 1280×720 px: una cuadrícula de 3 columnas $\times$ 2 filas, cada cuadro de $\approx 427 \times 360$ px.
`trim=\{izq\ inf\ der\ sup\}` recorta esos cuatro márgenes (en pt) del **mismo** archivo para aislar un cuadro.

Fórmula del recorte:

i Con ancho W , alto H , n_c columnas, n_f filas, y el cuadro en columna i , fila j (desde 0):

$$\text{trim} = \left\{ i \frac{W}{n_c}, (n_f - 1 - j) \frac{H}{n_f}, (n_c - 1 - i) \frac{W}{n_c}, j \frac{H}{n_f} \right\} \text{ pt}$$

! Ajusten estos valores a la resolución real de **su** imagen; lo importante es el método, no estos números exactos.

Código real: 6 subfiguras con trim

Código real que produce la figura compuesta (paquete `subfigure`, ya cargado por la clase):

```
\begin{figure}[H]
\centering
\subfigure[Reposo.]{\includegraphics
  [trim={0pt 360pt 853pt 0pt},clip,width=0.28\linewidth]{secuenciadefotos}
  \label{fig:seq-a}}
\subfigure[Entra.]{\includegraphics
  [trim={427pt 360pt 427pt 0pt},clip,width=0.28\linewidth]{secuenciadefotos}
  \label{fig:seq-b}}
\subfigure[Cruza izq.]{\includegraphics
  [trim={853pt 360pt 0pt 0pt},clip,width=0.28\linewidth]{secuenciadefotos}
  \label{fig:seq-c}}\
\subfigure[Gira.]{\includegraphics
  [trim={0pt 0pt 853pt 360pt},clip,width=0.28\linewidth]{secuenciadefotos}
  \label{fig:seq-d}}
\subfigure[Cruza der.]{\includegraphics
  [trim={427pt 0pt 427pt 360pt},clip,width=0.28\linewidth]{secuenciadefotos}
  \label{fig:seq-e}}
\subfigure[Sale.]{\includegraphics
  [trim={853pt 0pt 0pt 360pt},clip,width=0.28\linewidth]{secuenciadefotos}
  \label{fig:seq-f}}
\caption{Secuencia de movimiento sobre el agua.}
\label{fig:secuencia}
\end{figure}
```

Resultado visual equivalente (compilado)

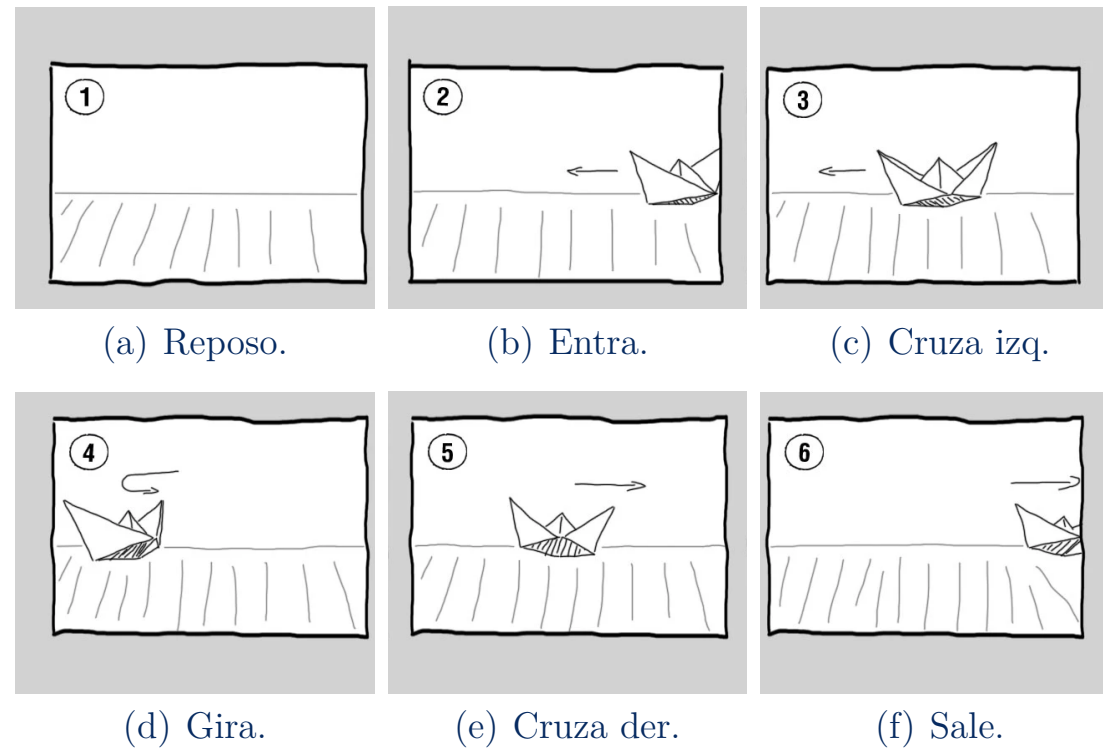


Figura: Secuencia de movimiento sobre el agua.

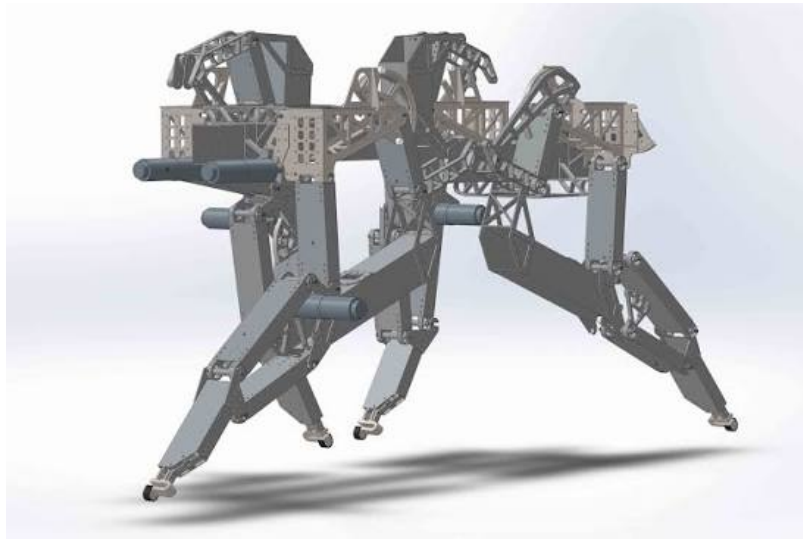


Así se cita: “Figure 1 shows the six stages of the maneuver; Fig. 1(a)–(c) correspond to the approach and Fig. 1(d)–(f) to the return.”

Regla 1: sin fondo en renders y capturas

Una captura de pantalla o un render 3D casi siempre incluye, sin que se den cuenta, el escritorio, la barra de tareas o un fondo degradado del software. Eso **no** se publica: distrae del contenido y se ve poco profesional.

No permitida – con fondo



Permitida – sin fondo

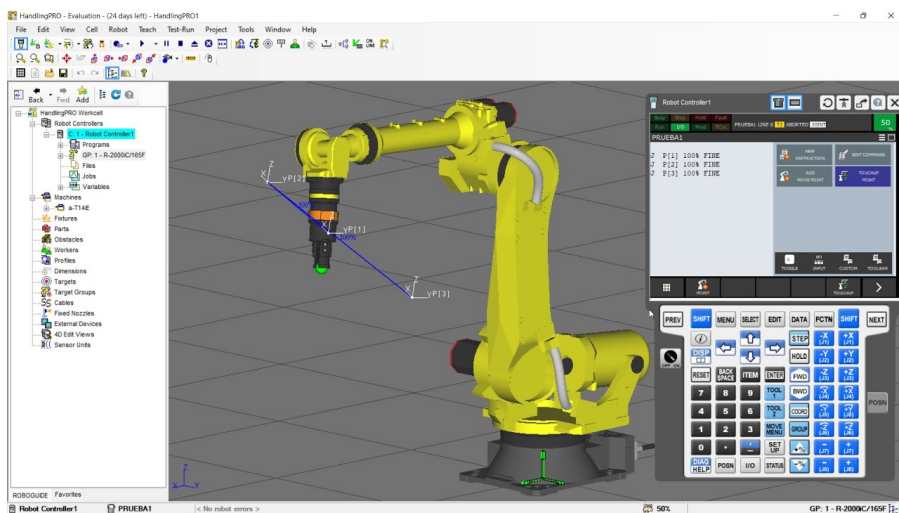


- ! Antes de exportar: usen fondo blanco o transparente en el software de render/simulación, o recorten el fondo en un editor de imágenes. El sujeto de la figura – no su entorno – es lo que debe verse.

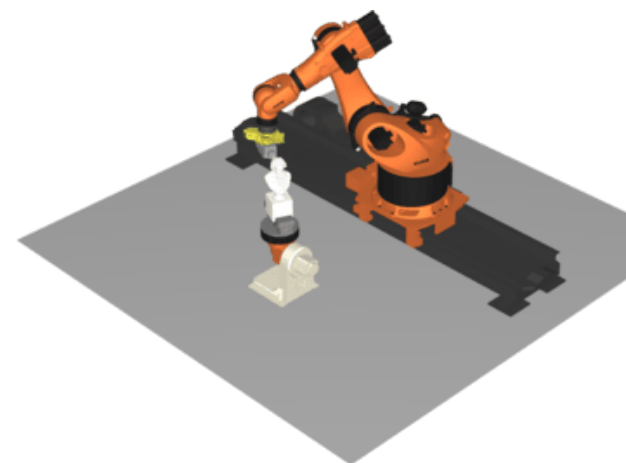
Regla 2: el marco del software, ¿sí o no?

Mostrar la ventana completa del software (menús, barras, teclado virtual) **casi nunca aporta información**: es ruido visual que compite con el contenido de la simulación.

No permitida – con marco



Permitida – sin marco



Excepción: si el *propósito explícito* de la figura es presentar la herramienta de trabajo (p.ej. en *Materials and Methods*, para mostrar con qué simularon), sí se justifica mostrar el marco del software. Fuera de ese caso, recorten al contenido.

Reglas y consideraciones de figuras

Antes de insertar una figura:

i

- ▶ Fondo blanco o transparente; nunca escritorio ni degradados del software.
- ▶ Sin marco de software, salvo que la figura exista para mostrar la herramienta.
- ▶ Misma resolución y estilo entre todos los cuadros de una figura compuesta.
- ▶ Etiqueten (`\label{}`) cada subfigura al crearla, no después.

Al citarla en el texto:

i

- ▶ Toda figura se menciona en el texto **antes** de mostrarse.
- ▶ El `\caption` debe entenderse sin leer el cuerpo del texto.
- ▶ Usen la referencia local para un cuadro (*Fig. 2(c)*) y la global para el conjunto (*Fig. 2*).
- ▶ Varias imágenes de una misma historia van en **una** figura compuesta, no en figuras sueltas.

De código a pseudocódigo

Su simulación corre en Python, MATLAB o C++, pero el paper **no debe mostrar ese código**: la comunidad científica documenta el algoritmo en **pseudocódigo**, independiente del lenguaje, con notación matemática consistente con el resto del artículo.

¿Por qué pseudocódigo y no el código fuente?:

- ▶ Es **reproducible en cualquier lenguaje**: describe la idea, no la sintaxis de Python.
- ▶ Usa **las mismas variables** que sus ecuaciones (Sección *Materials and Methods*), no nombres de variables de programación.
- ▶ Es el estándar de facto en revistas de ingeniería (IEEE, entre otras).

Agregar la librería al documento

En `plantilla\_IEEE\_LabDST.tex`, agreguen estos dos paquetes junto a los demás `\usepackage`, después de `\documentclass\{...\}\{IEEEtran\}`:

```
\usepackage{algorithm}
\usepackage{algpseudocode}
```

- ▶ `algorithm`: crea el **flotante** (como `figure` o `table`): numera, agrega `\caption` y admite `\label`.
- ▶ `algpseudocode`: da los **comandos** del pseudocódigo (`\State`, `\If`, `\For`, `\Return`, ...).

! Igual que con `Figure~\ref\{\}` y `Table~\ref\{\}`, un algoritmo se referencia con `Algorithm~\ref\{\}`.

Anatomía de un algoritmo en L^AT_EX

```
\begin{algorithm}[H]
\caption{Nombre del algoritmo}
\label{alg:etiqueta}
\begin{algorithmic}[1]
\Require entradas
\Ensure salidas
\State asignación $\gets$ valor
\If{condición}
    \State ...
\Else
    \State ...
\EndIf
\For{$k=1$ \textbf{to} $N$}
    \State ...
\EndFor
\State \Return resultado
\end{algorithmic}
\end{algorithm}
```

Comandos clave:

- ▶ `\Require`/`\Ensure`: entradas/salidas.
- ▶ `\State`: una línea de asignación o acción.
- ▶ `\If`/`\Else`/`\EndIf`: decisión.
- ▶ `\For`/`\EndFor`: repetición con contador.
- ▶ `\Call\{Nombre\}\{args\}`: llamada a otro algoritmo.
- ▶ `\Return`: valor de salida.

Ejemplo 1 – Saturación del ángulo del servo

Código (Python)

```
1 def clamp_angle(q, qmin, qmax):  
2     if q < qmin:  
3         return qmin  
4     elif q > qmax:  
5         return qmax  
6     else:  
7         return q
```

Pseudocódigo

Require: angle q , limits $q_{\min}$, $q_{\max}$

Ensure: clamped angle q'

```
1: if  $q < q_{\min}$  then  
2:    $q' \leftarrow q_{\min}$   
3: else if  $q > q_{\max}$  then  
4:    $q' \leftarrow q_{\max}$   
5: else  
6:    $q' \leftarrow q$   
7: end if  
8: return  $q'$ 
```



El código dice *cómo* lo programaron en Python (con `if`/`elif`/`return`); el pseudocódigo dice *qué* decide el algoritmo, con q , $q_{\min}$ y $q_{\max}$ – las mismas variables del rango físico del servomotor MG90S de cada módulo.

Ejemplo 1 – pseudocódigo y su fuente en L^AT_EX

Pseudocódigo (compilado)

Algorithm 1 Servo angle saturation

Require: angle q , limits $q_{\min}$, $q_{\max}$

Ensure: clamped angle q'

```
1: if  $q < q_{\min}$  then  
2:    $q' \leftarrow q_{\min}$   
3: else if  $q > q_{\max}$  then  
4:    $q' \leftarrow q_{\max}$   
5: else  
6:    $q' \leftarrow q$   
7: end if  
8: return  $q'$ 
```

Código L^AT_EX fuente

```
\begin{algorithm}[H]  
\caption{Servo angle saturation}  
\label{alg:clamp}  
\begin{algorithmic}[1]  
\Require angle $q$,  
  limits $q_{\min}, q_{\max}$  
\Ensure clamped angle $q'$  
\If{$q < q_{\min}$}  
  \State $q' \gets q_{\min}$  
\ElsIf{$q > q_{\max}$}  
  \State $q' \gets q_{\max}$  
\Else  
  \State $q' \gets q$  
\EndIf  
\State \Return $q'$  
\end{algorithmic}  
\end{algorithm}
```



En el texto: “*Algorithm 1 summarizes the saturation rule applied to each joint of a SOFIA module.*”

Ejemplo 2 – Bucle de simulación

Código (Python)

```
8 def simulate(modules, p_traj):
9     poses = []
10    for p_t in p_traj:
11        frame = []
12        for m in modules:
13            q1, q2 = inverse_kinematics(
14                m, p_t)
15            frame.append((q1, q2))
16        poses.append(frame)
17    return poses
```

Pseudocódigo

Require: modules $\{m_i\}_{i=1}^{16}$, target trajectory $\{p_k\}_{k=1}^N$
Ensure: pose sequence $\{(q_1^i, q_2^i)_k\}$

- 1: poses $\leftarrow []$
- 2: **for** $k = 1$ **to** N **do**
- 3: frame $\leftarrow []$
- 4: **for** $i = 1$ **to** 16 **do**
- 5: $(q_1^i, q_2^i) \leftarrow \text{INVERSEKINEMATICS}(m_i, p_k)$
- 6: APPEND(frame, (q_1^i, q_2^i))
- 7: **end for**
- 8: APPEND(poses, frame)
- 9: **end for**
- 10: **return** poses

! El doble `for` de Python (tiempo, luego módulo) es un `\For...\EndFor` anidado en pseudocódigo; la llamada a `inverse\_kinematics(...)` se vuelve `\Call\{InverseKinematics\}\{...\}`, referenciando el Algoritmo del Ejemplo 3.

Ejemplo 2 – pseudocódigo y su fuente en L^AT_EX

Pseudocódigo (compilado)

Algorithm 2 Tracking simulation loop

Require: modules $\{m_i\}_{i=1}^{16}$, target trajectory $\{p_k\}_{k=1}^N$

Ensure: pose sequence $\{(q_1^i, q_2^i)_k\}$

```
1: poses  $\leftarrow []$ 
2: for  $k = 1$  to  $N$  do
3:   frame  $\leftarrow []$ 
4:   for  $i = 1$  to 16 do
5:      $(q_1^i, q_2^i) \leftarrow \text{INVERSEKINEMATICS}(m_i, p_k)$ 
6:     APPEND(frame,  $(q_1^i, q_2^i)$ )
7:   end for
8:   APPEND(poses, frame)
9: end for
10: return poses
```

Código L^AT_EX fuente

```
\begin{algorithm}[H]
\caption{Tracking simulation loop}
\label{alg:sim}
\begin{algorithmic}[1]
\Require modules  $\{m_i\}$ ,
trajectory  $\{p_k\}$ 
\Ensure pose sequence
\State poses  $\gets []$ 
\For{$k=1$ \textbf{to} $N$}
\State frame  $\gets []$ 
\For{$i=1$ \textbf{to} $16$}
\State  $(q_1^i, q_2^i) \gets$ 
\Call{InverseKinematics}{$m_i, p_k$}
\State \Call{Append}{frame, ...}
\EndFor
\State \Call{Append}{poses, frame}
\EndFor
\State \Return poses
\end{algorithmic}
\end{algorithm}
```

Ejemplo 3 – Cinemática inversa de un módulo SOFIA

Este es el algoritmo central de su asignación: el mismo modelo de la sesión *Modelado cinemático*, con q_1, q_2 las variables articulares de los dos servos y $[u_x, u_y, u_z]^\top = \hat{r}_{2,p}$ la dirección unitaria hacia el punto p a seguir.

Código (Python)

```
18 def inverse_kinematics(m, p):
19     r2p = p - m.r02
20     u = r2p / norm(r2p)
21     q2 = asin(u[0])
22     q1 = atan2(-u[1], u[2])
23     q1 = clamp_angle(q1, Q1MIN, Q1MAX)
24     q2 = clamp_angle(q2, Q2MIN, Q2MAX)
25     return q1, q2
```

Pseudocódigo

Require: $r_{0,2}$, target p , joint limits

Ensure: q_1, q_2

- 1: $r_{2,p} \leftarrow p - r_{0,2}$
- 2: $[u_x, u_y, u_z]^\top \leftarrow r_{2,p} / \|r_{2,p}\|$
- 3: $q_2 \leftarrow \arcsin(u_x)$
- 4: $q_1 \leftarrow \text{atan2}(-u_y, u_z)$
- 5: $q_1 \leftarrow \text{CLAMP_ANGLE}(q_1, q_{1,\text{mín}}, q_{1,\text{máx}})$
- 6: $q_2 \leftarrow \text{CLAMP_ANGLE}(q_2, q_{2,\text{mín}}, q_{2,\text{máx}})$
- 7: **return** q_1, q_2

Ejemplo 3 – pseudocódigo y su fuente en L^AT_EX

Pseudocódigo (compilado)

Algorithm 3 SOFIA module inverse kinematics

Require: $r_{0,2}$, target p , joint limits

Ensure: q_1, q_2

- 1: $r_{2,p} \leftarrow p - r_{0,2}$
 - 2: $[u_x, u_y, u_z]^T \leftarrow r_{2,p} / \|r_{2,p}\|$
 - 3: $q_2 \leftarrow \arcsin(u_x)$
 - 4: $q_1 \leftarrow \operatorname{atan2}(-u_y, u_z)$
 - 5: $q_1 \leftarrow \operatorname{CLAMP_ANGLE}(q_1, q_{1,\text{mín}}, q_{1,\text{máx}})$
 - 6: $q_2 \leftarrow \operatorname{CLAMP_ANGLE}(q_2, q_{2,\text{mín}}, q_{2,\text{máx}})$
 - 7: **return** q_1, q_2
-

Código L^AT_EX fuente

```
\begin{algorithm}[H]
\caption{SOFIA module
  inverse kinematics}
\label{alg:ik}
\begin{algorithmic}[1]
\Require  $r_{0,2}$ $, target  $p$ $,
  joint limits
\Ensure  $q_1, q_2$ 
\State  $r_{2,p}$  \gets  $p - r_{0,2}$ 
\State  $[u_x, u_y, u_z]^{\top}$  \gets
   $r_{2,p} / \|r_{2,p}\|$ 
\State  $q_2$  \gets  $\arcsin(u_x)$ 
\State  $q_1$  \gets
   $\operatorname{atan2}(-u_y, u_z)$ 
\State  $q_1$  \gets$ \Call{ClampAngle}
  { $q_1, q_{1,\text{mín}}, q_{1,\text{máx}}$ }
\State  $q_2$  \gets$ \Call{ClampAngle}
  { $q_2, q_{2,\text{mín}}, q_{2,\text{máx}}$ }
\State \Return  $q_1, q_2$ 
\end{algorithmic}
\end{algorithm}
```

Reglas y consideraciones de pseudocódigo

Al escribirlo:

i

- ▶ Usen los comandos estándar (`\Require`, `\State`, `\If`, `\For`, `\Return`); no inventen sintaxis propia.
- ▶ Notación matemática consistente con sus ecuaciones (q_1 , q_2 , u_x , u_y , u_z), **no** nombres de variables de código (`self`, `q1\_servo`).
- ▶ Nada de sintaxis de un lenguaje (`def`, `print`, `;`): el pseudocódigo debe poderse reimplementar en cualquier lenguaje.

Al citarlo:

i

- ▶ Etiqueten con `\label{}` dentro del `algorithm`, junto al `\caption`.
- ▶ Cítenlo en el texto **antes** de que aparezca: “*Algorithm 3 computes the joint angles...*”
- ▶ Si un algoritmo llama a otro (`\Call{ }{ }`), referencien ambos números en el texto.

La receta completa, en una subsección

Una subsección de resultados de simulación bien escrita combina las tres piezas en este orden: se anuncia el algoritmo, se anuncia la figura, y ambos se citan en el texto que los conecta.

Plantilla de redacción:

“Algorithm 3 shows the inverse-kinematics update executed by each module. Using this update, Figure 1 shows six frames extracted from the tracking video; Fig. 1(a)–(c) show the point entering the module’s workspace and Fig. 1(d)–(f) show the platform re-orienting to keep $r_{2,3}$ aligned with it. [Resultado numérico u observación breve.]”

! Tres a cinco líneas bastan. El texto no repite lo que la figura o el algoritmo ya muestran: los **conecta** y añade lo que ninguno de los dos dice por sí solo (el resultado, la comparación, la conclusión).

Sesión 4 · De la simulación al paper, Asign.1

Redacten la subsección de *Results* (o *Design and Implementation*) del artículo de LabDST-Sofia correspondiente a la **simulación del panel de 16 módulos SOFIA siguiendo un punto móvil, aplicando la cinemática directa e inversa** de la sesión *Modelado cinemático*.

- (a) **Figura compuesta:** al menos 4 cuadros extraídos del video de su simulación (distintos instantes de t , o distintos módulos siguiendo el punto), con `figure`+`subfigure`, sin fondo ni marco de software, y con `\label` local y global.
- (b) **Pseudocódigo:** el algoritmo de cinemática inversa de un módulo que implementaron (basado en $\arcsin/\arctan2$ y la saturación de ángulos de la sesión técnica), con `algorithm`+`algorithmic` y su `\label`.
- (c) **Texto breve:** un párrafo que cite ambos –antes de que aparezcan– y narre qué se simuló, qué modelo se aplicó (cinemática directa/inversa) y qué muestra la figura.